Web Technologies

Background Image

Grid Layout

CSS Responsiveness

Background Image

- Background-image : url(‘link to image’)
- Background size: cover/contain/auto (by default)
- Background-repeat: no-repeat
- Background-position: center
- In Tailwind:
  - bg-[url('Unknown')] bg-contain bg-no-repeat

Grid Layout

- The CSS Grid Layout Module offers a grid-based layout system, with rows and columns, making it easier to design web pages without having to use floats and positioning.
- A grid layout consists of a **parent** element, with **one or more child elements.**
- To make an HTML element behave as a grid container, you have to set the display property to grid or inline-grid.
- Grid containers consist of grid items, placed inside columns and rows.

```
.grid-container {
  display: grid;
  grid-template-columns: auto auto auto auto;
}
```

```
.grid-container {
  display: grid;
  grid-template-columns: 80px 200px auto 40px;
}
```

The grid-template-[columns/rows] Property

- The grid-template-columns property defines the number of columns in your grid layout, and it can define the width of each column.
- The value is a space-separated-list, where each value defines the width of the respective column.
- If you want your grid layout to contain 4 columns, specify the width of the 4 columns, or "auto" if all columns should have the same width.
- The grid-template-rows property defines the height of each row.

```css
.grid-container {
  display: grid;
  grid-template-columns: auto auto auto;
  grid-template-rows: 80px 200px;
  gap: 10px;
  background-color: #2196F3;
  padding: 10px;
}
```

```html
<body>

<h1>The grid-template-rows Property</h1>

<div class="grid-container">
  <div>1</div>
  <div>2</div>
  <div>3</div>
  <div>4</div>
  <div>5</div>
  <div>6</div>
</div>

<p>Use the <em>grid-template-rows</em> property to specify the size
(height) of each row.</p>

</body>
</html>
```

The grid-template-rows Property

1	2	3
4	5	6

Use the *grid-template-rows* property to specify the size (height) of each row.

Justify-content and align-items

- The justify-content and align-items work with grid layout, as well. The

```
<style>
.grid-container {
  display: grid;
  height: 300px;
  justify-content: center;
  align-content: center;
  grid-template-columns: 50px 50px 50px; /*Make the grid smaller than the
container*/
  gap: 10px;
  background-color: #2196F3;
  padding: 5px;
}

.grid-container > div {
  background-color: rgba(255, 255, 255, 0.8);
  text-align: center;
  padding: 20px 0;
  font-size: 30px;
}
</style>
```

CSS Grid Item

- A grid container contains grid items.
- By default, a container has one grid item for each column, in each row, but you can style the grid items so that they will span multiple columns and/or rows.
- The grid-column property defines on which column(s) to place an item.
- You define where the item will start, and where the item will end.
- The grid-column property is a shorthand property for the grid-column-start and the grid-column-end properties.
- To place an item, you can refer to line numbers, or use the keyword "span" to define how many columns the item will span.

```
<style>
.grid-container {
  display: grid;
  grid-template-columns: auto auto auto auto auto auto;
  gap: 10px;
  background-color: #2196F3;
  padding: 10px;
}

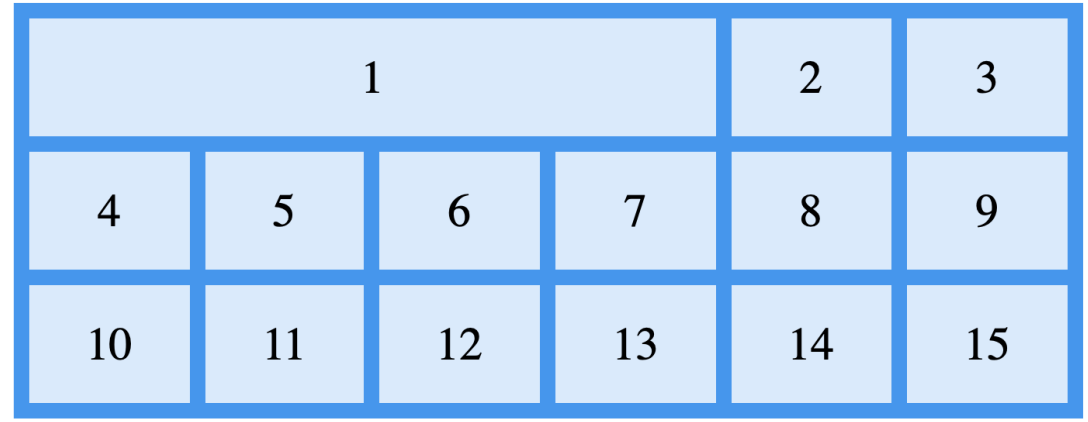
.grid-container > div {
  background-color: rgba(255, 255, 255, 0.8);
  text-align: center;
  padding: 20px 0;
  font-size: 30px;
}

.item1 {
  grid-column: 1 / 5;
}
</style>
```

The grid-column Property

Use the *grid-column* property to specify where to place an item.

Item1 will start on column 1 and end before column 5:



```
<style>
.grid-container {
  display: grid;
  grid-template-columns: auto auto auto auto auto auto;
  gap: 10px;
  background-color: #2196F3;
  padding: 10px;
}

.grid-container > div {
  background-color: rgba(255, 255, 255, 0.8);
  text-align: center;
  padding: 20px 0;
  font-size: 30px;
}

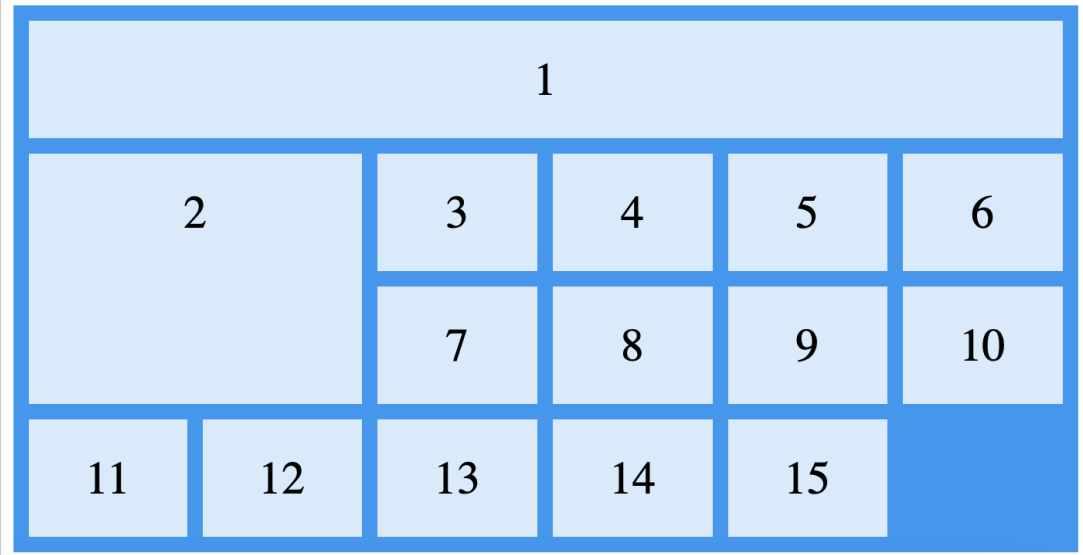
.item1 {
  grid-column: 1 / 7;
}
.item2 {
  grid-column: 1 / 3;
  grid-row: 2 / 4;
}

</style>
```

The grid-column Property

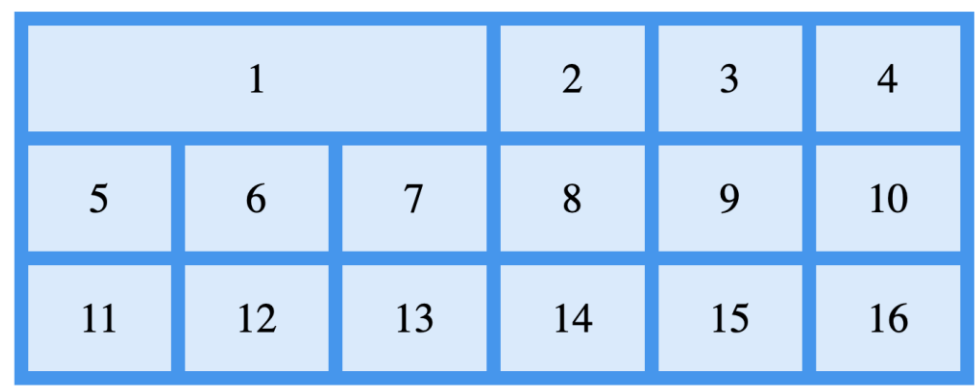
Use the *grid-column* property to specify where to place an item.

Item1 will start on column 1 and end before column 5:



Make "item1" start on column 1 and span 3 columns:

```
.item1 {
  grid-column: 1 / span 3;
}
```



The grid-area Property

- The grid-area property can be used as a shorthand property for the grid-row-start, grid-column-start, grid-row-end and the grid-column-end properties.
- Make "item8" start on row-line 1 and column-line 2, and end on row-line 5 and column line 6:

```
.item8 {
  grid-area: 1 / 2 / 5 / 6;
}
```

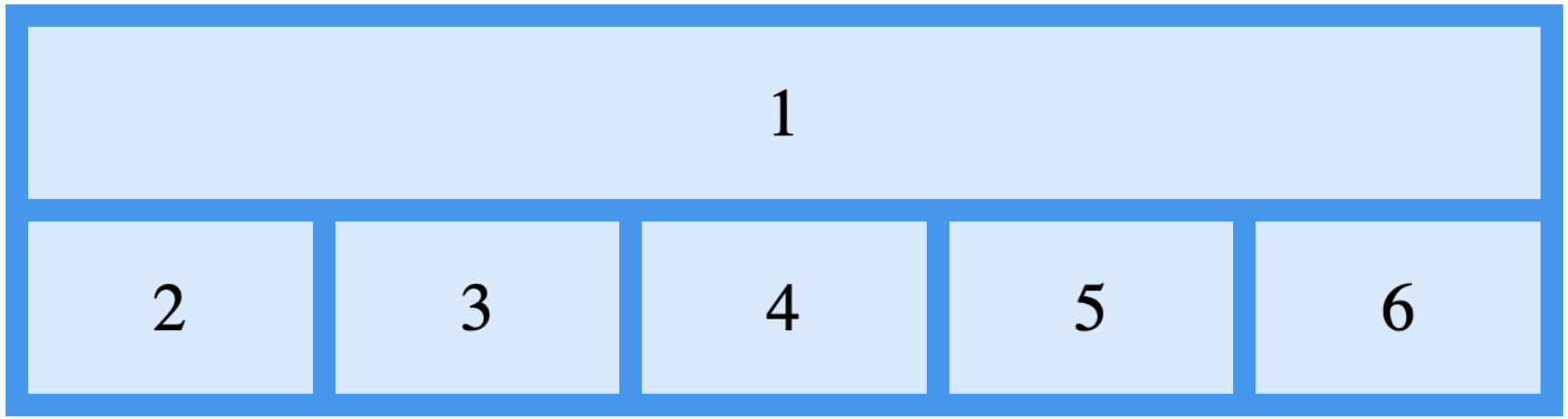
- Make "item8" start on row-line 2 and column-line 1, and span 2 rows and 3 columns:

```
.item8 {
  grid-area: 2 / 1 / span 2 / span 3;
}
```

The grid-area Property

- The grid-area property can also be used to assign names to grid items.
- Item1 gets the name "myArea" and spans all five columns in a five columns grid layout:

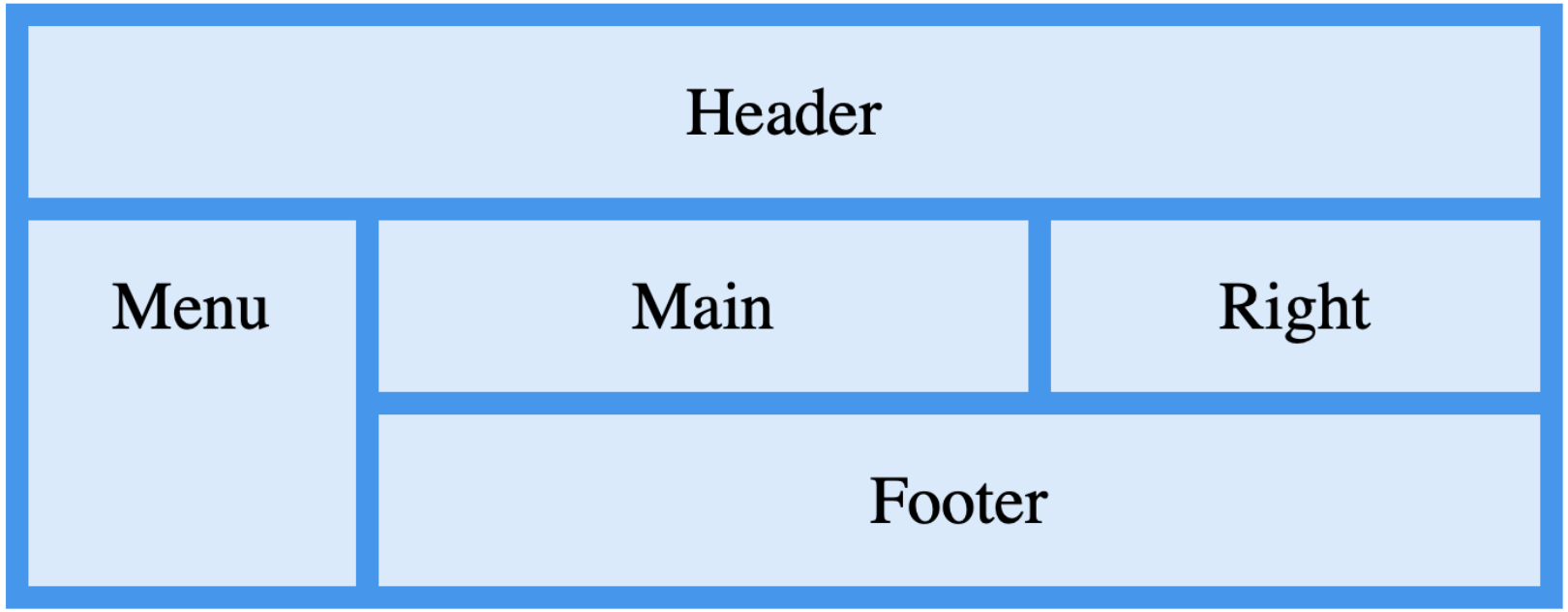
```
.item1 {
  grid-area: myArea;
}
.grid-container {
  grid-template-areas: 'myArea myArea myArea myArea myArea';
}
```



```
<style>
.item1 { grid-area: header; }
.item2 { grid-area: menu; }
.item3 { grid-area: main; }
.item4 { grid-area: right; }
.item5 { grid-area: footer; }

.grid-container {
  display: grid;
  grid-template-areas:
    'header header header header header header'
    'menu main main main right right'
    'menu footer footer footer footer footer';
  gap: 10px;
  background-color: #2196F3;
  padding: 10px;
}

.grid-container > div {
  background-color: rgba(255, 255,
  text-align: center;
  padding: 20px 0;
  font-size: 30px;
}
</style>
```



CSS Units

- CSS has several different units for expressing a length.
- Many CSS properties take "length" values, such as width, margin, padding, font-size, etc.
- Length is a number followed by a length unit, such as 10px, 2em, etc.
- Set different length values, using px (pixels):
- There are two types of length units:
  - **absolute** and **relative**.

```
h1 {
  font-size: 60px;
}

p {
  font-size: 25px;
  line-height: 50px;
}
```

Absolute Lengths

- The absolute length units are fixed and a length expressed in any of these will appear as exactly that size.
- Absolute length units are not recommended for use on screen, because screen sizes vary so much. However, they can be used if the output medium is known, such as for print layout.

Unit	Description
cm	centimeters
mm	millimeters
in	inches (1in = 96px = 2.54cm)
px *	pixels (1px = 1/96th of 1in)
pt	points (1pt = 1/72 of 1in)
pc	picas (1pc = 12 pt)

Relative Lengths

Relative length units specify a length relative to another length property.
Relative length units scale better between different rendering medium.

Unit	Description
em	Relative to the font-size of the element (2em means 2 times the size of the current font)
ex	Relative to the x-height of the current font (rarely used)
ch	Relative to the width of the "0" (zero)
rem	Relative to font-size of the root element
vw	Relative to 1% of the width of the viewport*
vh	Relative to 1% of the height of the viewport*
vmin	Relative to 1% of viewport's* smaller dimension
vmax	Relative to 1% of viewport's* larger dimension
%	Relative to the parent element

Media Queries

- Help create a responsive website

```
@media (width:600px) {
    div {
        background-color: black;
    }
}
@media (min-width:600px) {
    div {
        background-color: cadetblue;
        width: 300px;
        height: 300px;
    }
}
@media (min-width:1000px) {
    div {
        background-color: rgb(158, 95, 160);
        width: 500px;
        height: 500px;
    }
}
```

Mobile first approach

- Design for mobile, and then gradually include bigger devices.

```
<style>
    div {
        background-color: aqua;
        width: 100px;
        height: 100px;
    }
    @media (min-width:600px) and (max-width:1000px){
        div {
            background-color: black;
            color: white;
        }
    }
    @media (min-width:1000px) and (max-width:1400px){
        div {
            background-color: rgb(127, 73, 132);
        }
    }
</style>
```

Breakpoints

- There are tons of screens and devices with different heights and widths, so it is hard to create an exact breakpoint for each device. To keep things simple you could target five groups:
- ```
  /* Extra small devices (phones, 600px and down) */
  @media only screen and (max-width: 600px) {...}

  /* Small devices (portrait tablets and large phones, 600px and up) */
  @media only screen and (min-width: 600px) {...}

  /* Medium devices (landscape tablets, 768px and up) */
  @media only screen and (min-width: 768px) {...}

  /* Large devices (laptops/desktops, 992px and up) */
  @media only screen and (min-width: 992px) {...}

  /* Extra large devices (large laptops and desktops, 1200px and up) */
  @media only screen and (min-width: 1200px) {...}
  ```
  ```

Container Queries

- Container queries allow you to **apply styles** based on the **size of an element's parent container**, rather than the size of the viewport (which is what media queries do).
- This makes components more **flexible** because they can **adapt dynamically inside any layout**, not just based on screen size.
- Unlike media queries, which depend on the entire screen width, **container queries allow components to adapt to their container size**.
- Example: A card may look different in a sidebar vs. the main content area.

Container Queries

If you want the card component to have different style based on the size of it’s container, then wrap it inside a parent container, with container-type set to inline-size

```html
<h1>Container Query</h1>
<div class="wrapper">
    <div class="card">
        <h1>Card1</h1>
        <p>Lorem ipsum dolor sit, amet consectetur
        adipisicing elit. Atque, voluptatibus.</p>
    </div>
</div>
```

Here, the styles will apply, once the parent div reaches 300px.

Container Query

Card1

Lorem ipsum dolor sit, amet consectetur adipisicing elit. Atque, voluptatibus.

At 300px

```
.wrapper {
    container-type: inline-size;
    width: 50%;
    padding: 10px;
    background: lightgray;
}

.card {
    width: 80%;
    padding: 10px;
    background-color: lightgrey;
}

@container (min-width: 300px) {
    .card {
        background: lightblue;
        font-size: 20px;
    }
}
```

Activity

- Create a blog website with a single column view for mobile and a 3 column view for laptops/desktops.